# Error Control Codes save Distributed Gradient Descent

Joseph Shenouda

April 2020

## 1 Introduction

As optimization becomes more and more ubiquitous these days (especially in machine learning) new solutions are arising to make these algorithms not only more accurate but also more computationally feasible. Consider the following optimization problem:

$$\min_{\beta} \quad \sum_{i=1}^{D} \ell(\beta; x_i, y_i) \tag{1}$$

We attempt to minimize the loss function by finding the best $\beta$ based on our data samples $(x_i, y_i)$. A very common approach that is widely used is the famous Gradient Descent algorithm, this algorithm works in an iterative fashion updating $\beta$ by moving in the direction of the negative gradient.

$$\beta_{k+1} = \beta_k - \nabla \sum_{i=1}^{D} \ell(\beta_k; x_i, y_i) \tag{2}$$

However as our number of data samples $D$ increases computing the gradient becomes more computationally expensive. Thankfully due to the additive separable nature of the optimization problems commonly encountered in machine learning like the one presented in (1) these problems lend themselves very well to parallelization. Notice that we can express the gradient of our loss function as follows:

$$\nabla \sum_{i=1}^{D} \ell(\beta_k; x_i, y_i) = \nabla \sum_{i=1}^{D_1} \ell(\beta_k; x_i, y_i) + \nabla \sum_{i=1}^{D_2} \ell(\beta_k; x_i, y_i) + ... + \nabla \sum_{i=1}^{D_n} \ell(\beta_k; x_i, y_i) \tag{3}$$

Where $D_1 ... D_n$ are partitions of our total data evenly split into $n$ partitions. We could then setup a distributed setting with a master node and $n$ workers where each worker gets a data partition $D_i$ and computes a partial gradient. Then once all workers have finished computing they send it back down to the master which will aggregate the gradients and take the gradient step.

However it is often the case that not all workers compute their gradient at the same rate thus we are presented with a "straggler" effect where any $s (s < n)$ nodes can slow down the whole operation because the master must wait until all have finished in order to get an accurate gradient. For the rest of this paper we will assume that any $s$ straggler nodes are slow up to failure meaning we never obtain their respective gradient. This is analogous to an erasure when we try to transmit a codeword over some channel where we may never receive the missing bit but must somehow recover the whole message. Coding theorists know that this problem can usually be solved by introducing some clever redundancy into the problem and the trade off is always between extra redundancy that increases robustness and the communication overhead introduced by adding this redundancy. We present an analogous framework here.

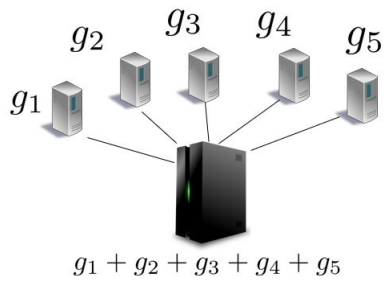


Figure 1: Example of distributed gradient descent with 5 workers

## 2 Gradient Coding Framework

Consider the case where we have $n = 3$ workers and up to $s = 1$ straggler. The following scheme in Figure 2 can be obtained based on a framework known as "Gradient Coding" (Tandon et al., "Gradient Coding."). Notice that if any $s = 1$ worker is a straggler the master can still recover the full gradient by taking a linear combination of the vector sent down from any 2 workers. We can clearly see how this setup works for our small example and now we will see how to generalize this framework and apply it to any $n$ workers subject to up to $s$ stragglers.

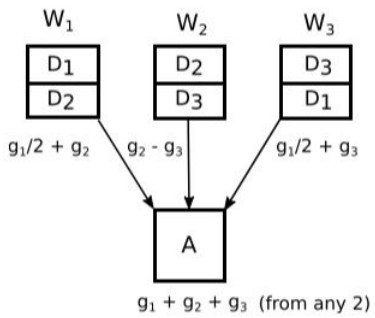


Figure 2: Source: Tandon et al., "Gradient Coding."

To begin we will assume that if we have a dataset with $D$ total samples we will divide it into $n$ equally size data partitions of size $\frac{d}{n}$. We will introduce 2 matrices that define how the data partitions are distributed and what linear combination of the partial gradient each worker will compute as well as how to recover the full gradient from any $n - s$ workers. Consider the matrix $B$, this matrix will encode what data partitions each worker gets and what linear combination of partial gradient it computes. Each row of $B$ denoted $b_i$ corresponds to the $i^{th}$ worker in our system so from our above example $B\mathbf{g}$ is:

$$\begin{pmatrix} 1/2 & 1 & 0 \\ 0 & 1 & -1 \\ 1/2 & 0 & 1 \end{pmatrix} \begin{pmatrix} -g_1- \\ -g_2- \\ -g_3- \end{pmatrix} = \begin{pmatrix} g_1/2 + g_2 \\ g_2 - g_3 \\ g_1/2 + g_3 \end{pmatrix} \tag{4}$$

Notice that $\mathbf{1}_{1\times 3}$ is in the span of any 2 rows of $B$ this will be important as we introduce the second matrix.

So now that we have this matrix $B \in \mathbf{R}^{n\times n}$ we can see how we describe the distribution of the data. Next we must explain how the master node can recover $g_1 + g_2 + g_3$ or in general $g_1 + g_2 + .. + g_n$. We can describe this

by a matrix $A$ that will encapsulate different failure scenarios. Each row of $A$ denoted $a_j$ tells us which nodes are straggling by assigning a 0 to the $i^{th}$ column of $a_j$ here each column of $A$ represents a worker. Since we have $n$ nodes and $s$ stragglers we have $f = \binom{n}{s}$ different failure scenarios. For our example we now have

$$A = \begin{pmatrix} 0 & 1 & 2 \\ 1 & 0 & 1 \\ 2 & -1 & 0 \end{pmatrix} \tag{5}$$

thus,

$$\begin{pmatrix} 0 & 1 & 2 \\ 1 & 0 & 1 \\ 2 & -1 & 0 \end{pmatrix} \begin{pmatrix} 1/2 & 1 & 0 \\ 0 & 1 & -1 \\ 1/2 & 0 & 1 \end{pmatrix} \mathbf{g} = \begin{pmatrix} g_1 + g_2 + g_3 \\ g_1 + g_2 + g_3 \\ g_1 + g_2 + g_3 \end{pmatrix} \tag{6}$$

which means

$$\mathbf{1g} = \begin{pmatrix} g_1 + g_2 + g_3 \\ g_1 + g_2 + g_3 \\ g_1 + g_2 + g_3 \end{pmatrix} \tag{7}$$

Thus we see that this problem really boils down to finding these 2 matrices $A \in \mathbf{R}^{f \times n}$ with a specific support structure and $B \in \mathbf{R}^{n \times n}$ such that $AB = \mathbf{1}_{n \times n}$.

Now we cannot just pick any 2 random matrices that satisfy these $AB = \mathbf{1}$ here we present some conditions that must be imposed on these matrices. We have seen already that $A$ must satisfy our condition that each row represents a different failure scenario based on which indices are non-zero. Also we alluded to the fact that the $B$ matrix must satisfy a very strict condition which is that any $n - s$ rows must have the $\mathbf{1_{1 \times n}}$ in its span. This is because in order to be robust to any $s$ stragglers we must be able to recover the total gradient from any surviving $n - s$ workers which we say is equivalent to getting $\mathbf{1_{n \times n}}$. Thus we have condition 1:

**Condition 1**: Consider the (A,B) scheme robust to any s stragglers given n workers we require that for any subset $I \subseteq [n]$ with $I = n - s$ (i.e. any $n - s$ rows) we must have that $\mathbf{1_{1 \times n}} \in span\{b_i | i \in I\}$

So now we know that any $B$ matrix satisfying this condition will allow us to be robust to any $s$ stragglers. What immediately comes to a person's mind might be to make this $B$ matrix contain all ones in each of its rows because we clearly know that $n - s$ rows of all ones has the all ones vector in its span. But notice that this would be problematic because if we have a matrix of all ones like we described it would mean that each worker gets all $n$ data partitions and computes the full gradient which completely defeats the purpose of distributed computing. So what we ideally want is to minimize as much as possible the number of non-zero elements in each row of $B$ so that each worker computes as few gradients as possible. In other words we want to minimize the $||b_i||_0$ for every $i^{th}$ row of $B$. (Note: the $||.||_0$ norm counts the number of non-zero values in a vector, often called the $\ell_0$ norm). This leads us to a lower bound on the sparsity of each row of $B$.

**Theorem 1.** *For any (A,B) scheme robust to any s straggler we must have:* $||b_i||_0 \geq (s + 1)$

This theorem gives us a lower bound on the density of $B$ telling us that even the best case scenario for $B$ has to have $||b_i||_0 = s + 1$ meaning each worker will get exactly $s + 1$ data partitions.

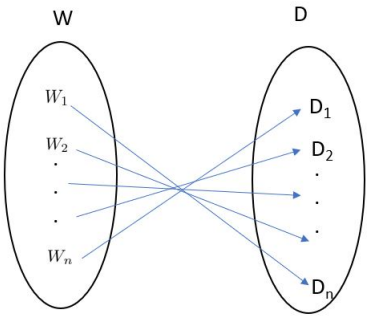


Figure 3: Bipartite graph between workers and data partitions

*Proof.* Consider the bipartite graph between the workers and each data partition. By definition the degree of each node $i$ in the worker set is the same as the $||b_i||_0$. Also the degree of each element in the data partitions set must have degree $\geq (s+1)$ because consider the case when a data partition is associated with only $s$ or less workers. This can result in never getting the gradient corresponding to that portion of the data in the event that all $s$ of those workers straggle. Being that this is a bipartite graph the sum of the degree of each element in $W$ set must equal the sum of the degree of all elements in the $D$ set.

$$\sum_{i=1}^{n} deg(W_i) = \sum_{i=1}^{n} deg(D_i)$$

thus since we know that the degree of every element in the $W$ is $||b_i||_0$ we have $\sum_{i=1}^{n} deg(W_i) = n||b_i||_0$ and since the degree of each element in the $D$ set must be $\geq (s+1)$ we know $\sum_{i=1}^{n} deg(D_i) \geq n(s+1)$ thus,

$$||b_i||_0 \geq (s+1)$$

□

This whole time we have only discussed conditions and bounds for our $B$ matrix giving no attention to the $A$ matrix. As it turns out given any $B$ matrix which the right conditions we can always find $A$ with the corresponding correct support structure such that $AB = \mathbf{1}$ the algorithm is provided in Algorithm 1 in Tandon et al., "Gradient Coding.". We will focus on the $B$ matrix for the rest of this paper and show how we can construct a $B$ fulfilling our lower bound using cyclic MDS codes.

# 3 Construction of B from Cyclic MDS Codes

We mentioned earlier that the best $B$ matrix would be the one that fulfills the lower bound we established ($||b_i||_0 = s+1$) to ensure that each worker does the least amount of redundant processing. Therefore our best $B$ matrix would look something like Figure 4. Where each row has exactly $s+1$ non-zeros.

Now the question is how can we construct a a $B$ with this specific structure while guaranteeing that it satisfies our Condition 1. In the Raviv et al., "Gradient Coding from Cyclic MDS Codes and Expander Graphs." paper they show how this can be done by having each column in $B$ be a codeword of a cyclic MDS code. First we need some important lemmas about MDS codes in general.

$$\begin{pmatrix} \beta_1 & 0 & \cdots & 0 & \beta_{s+1} & \beta_s & \cdots & \beta_2 \\ \beta_2 & \beta_1 & 0 & \cdots & 0 & \beta_{s+1} & \cdots & \beta_3 \\ \vdots & \vdots & \ddots & \ddots & \vdots & \ddots & \ddots & \vdots \\ \beta_s & \beta_{s-1} & \cdots & \beta_1 & 0 & \cdots & 0 & \beta_{s+1} \\ \beta_{s+1} & \beta_s & \cdots & \beta_2 & \beta_1 & 0 & \cdots & 0 \\ 0 & \beta_{s+1} & \cdots & \beta_3 & \beta_2 & \beta_1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & \beta_{s+1} & \beta_s & \beta_{s-1} & \cdots & \beta_1 \end{pmatrix}$$

Figure 4: Source: Raviv et al., "Gradient Coding from Cyclic MDS Codes and Expander Graphs."

**Lemma.** *If the code $C \subseteq \mathbf{F}^n$ is an $[n, k]$ MDS code, then*
*1. $C^\perp$ is an $[n, n-k]$ MDS code with minimum weight $k+1$ (Recall MDS codes meet Singleton bound)*
*2. The reverse code $C^R \triangleq \{(c_n, ..., c_1) | (c_1, ..., c_n) \in C\}$ is an $[n, k]$ MDS code.*
*3. For any subset $K \subseteq [n]$ of size $n-k+1$ there exists a $c \in C$ whose support is $K$.*

We will not go over the proof of the first 2 lemmas the first can be found in R. Roth, Introduction to coding theory. Cambridge University Press, 2006. Prob 4.1. The second lemma is true because permuting the elements in codewords will not change the minimum weight. For the third lemma it is helpful to understand what it is really saying through a simple example before moving on to the proof.

**Example 1.** *Supposed we have a $[5, 2]$ MDS code with $d_{min} = 5 - 2 + 1 = 4$*
*Now let choose $K = \{1, 3, 4, 5\}$ clearly a subset of $[n]$ and of cardinality 4, then the claim is that there exists a codeword $c \in C$ such that $c$ is of the form $(*, 0, *, *, *)$ where the asterisks denote non-zero elements.*

The proof is as follows

*Proof.* Consider the generator matrix of the $[n, k]$ code $G \in \mathbf{F}^{k \times n}$ and $K \subseteq [n]$ where $|K| = n - k + 1$. Now let $G_I \in \mathbf{F}^{k \times (k-1)}$ be a submatrix made up of the columns indexed by $K^c$ (the complement of $K$). Now since this matrix is not square we know that its rank is at most $(k-1)$ which implies that its left null space is non-zero. Therefore there exists $\mathbf{y} \in \mathbf{F}^{1 \times k}$ such that $\mathbf{y}G_I = (0, 0, 0....0)$. Now this means that $\mathbf{y}G$ is a codeword which has zeros all $K^c$ indices. Now to show that the remaining $n - k + 1$ elements of the codeword are non-zero recall that the minimum weight of any codeword is $n - k + 1$ therefore those remaining indices must be non-zero and those are indeed the indices from $K$. □

Now with these 3 lemmas on hand we can proceed to the main result. Let $C$ be an $[n, n-s]$ cyclic MDS code that contains $\mathbf{1}_{1 \times n}$ then we can construct a $B$ matrix as Figure 4, where each column of $B$ is a codeword of $C$ and each row of $B$ is a codeword of $C^R$. Finally and most importantly every set of $n - s$ rows of $B$ are linearly independent over $\mathbf{F}$ and therefore must contain $\mathbf{1_{1 \times n}}$ in their span satisfying our first condition. Now to conclude we will prove that every $n - s$ rows of $B$ are indeed linearly independent.

*Proof.* Assume for the sake of contradiction that there exists a set of $n - s$ linearly dependent rows. This would imply that there exists $\mathbf{v} \in \mathbf{F}^{1 \times n}$ with Hamming weight $n - s$ such that $\mathbf{v}B = 0$. Now since by construction all the columns of $B$ are codewords of $C$ this must imply that $\mathbf{v} \in C^\perp$. But from our lemmas above we know that the minimum weight of any $c \in C^\perp$ where $C$ is MDS is $n - s + 1$ which contradicts our statement that $\mathbf{v}$ with weight $n - s$ exists. □

# 4 Conclusion

In this paper we summarized some of the work done in the paper Tandon et al., "Gradient Coding." which set up our Gradient coding framework and presented the necessary conditions for our $B$ matrix. We also discussed the ideas presented in Raviv et al., "Gradient Coding from Cyclic MDS Codes and Expander Graphs." to show how $B$ can be constructed from a cyclic MDS code. Some other interesting and related topics which we did not cover are how to handle partial stragglers, workers that do not straggle to failure. We also did not consider using stochastic gradient descent which is very popular and does not need the whole dataset. In conclusion we saw how the basic ideas in error control coding can be applied to areas that seemingly are unrelated to the initial motivation for coding theory but upon some analysis we reframed the problem to be solved with some popular coding theory tools that we are very familiar with.

# 5 Refrences


1. Tandon, Rashish, Qi Lei, Alexandros G. Dimakis, and Nikos Karampatziakis. "Gradient Coding." ArXiv:1612.03301 [Cs, Math, Stat], March 7, 2017. http://arxiv.org/abs/1612.03301.

2. Raviv, Netanel, Itzhak Tamo, Rashish Tandon, and Alexandros G. Dimakis. "Gradient Coding from Cyclic MDS Codes and Expander Graphs." ArXiv:1707.03858 [Cs, Math, Stat], July 8, 2019. http://arxiv.org/abs/1707.03858.